

Технологии
Программирования

Будь как вода



質問

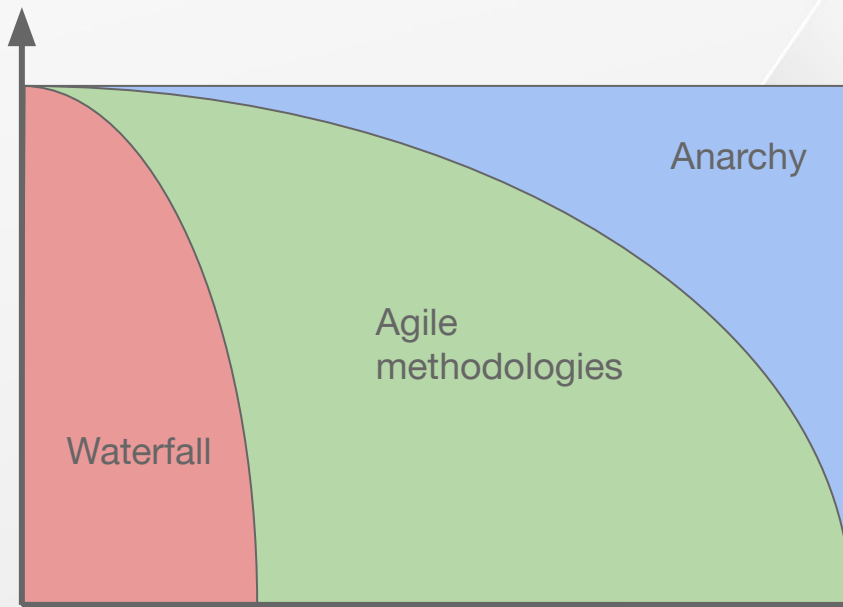
Стили управления проектами

- Waterfall
- Lean
 - SAFE
- Agile
 - Scrum
 - Kanban
 - Anarchy
 - SAFE
 - ...



世界地図

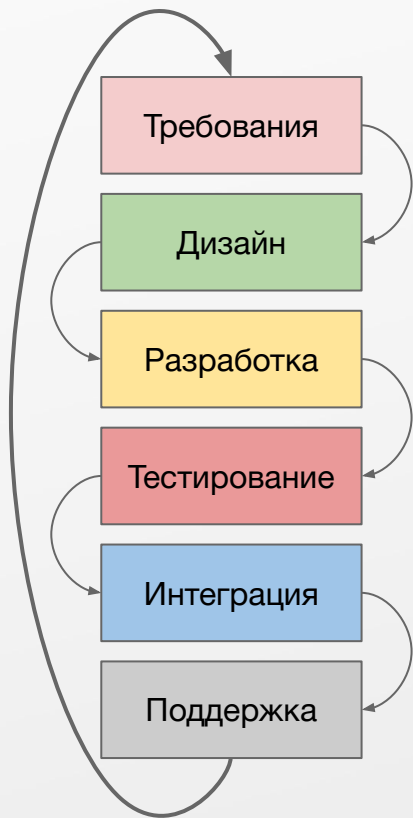
Сложность



Неопределенность



滝



Применимость:

- Полностью прозрачные требования,
- Заказчик отсутствует или не предоставляет информации,
- Экспертная лидерская команда,
- Большая команда.

Плюсы:

- Стабильность требований,
- Высокая предсказуемость,
- Низкая нагрузка на разработчиков.

Минусы:

- Длительная подготовка,
- Высокая нагрузка на лидеров команды,
- Требование завершения предыдущей фазы перед началом следующей,
- Отсутствие контакта с заказчиком,
- Высокий риск промаха по рынку.



守破離

Shuhari пришел из японских боевых искусств, наследник 序破急 johakyu:

- Shu - познай,
- Ha - разрушь,
- Ri - выдели.

Описывает процесс развития человека от новичка до мастера.

改善 Kaizen/LEAN

- Непрерывное совершенствование процессов производства,
- Цель кайдзен - бережливое производство без потерь 無駄 Muda:
 - 無理 Muri - потери избыточной нагрузки,
 - 斑 Mura - потери неравномерности нагрузки (затраты на лишние действия),
 - Максимальная эффективность без лишних маневров и бюрократии.
- Широко интегрирован в производство Toyota motors - LEAN.



**И да, это требует быстрого и точного выполнения инструкций.
Любое отклонение приводит к появлению 無駄 Muda.**



Agile Manifesto

Мы **постоянно открываем для себя более совершенные методы разработки** программного обеспечения, занимаясь разработкой непосредственно и помогая в этом другим.

Благодаря проделанной работе мы смогли осознать, что:

- **Люди и взаимодействие** важнее процессов и инструментов,
- **Работающий продукт** важнее исчерпывающей документации,
- **Сотрудничество с заказчиком** важнее согласования условий контракта,
- **Готовность к изменениям** важнее следования первоначальному плану.

То есть, не отрицая важности того, что справа,
мы всё-таки больше ценим то, что слева.

**Agile - не процесс разработки ПО, это методология формирования
процессов разработки ПО!**



影分身の術

Применимость:

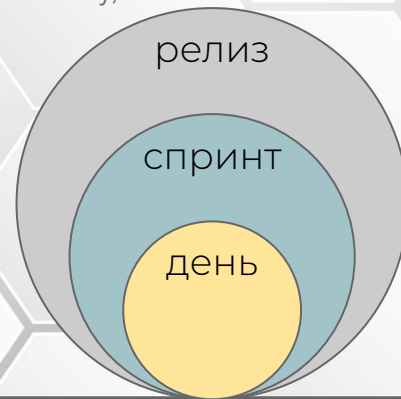
- Требований нет или они подвижны,
- Присутствует заказчик и готов предоставлять информацию и требования,
- Экспертная инженерная и лидерская команда,
- Маленькая команда (до 8 человек).

Плюсы:

- Короткая подготовка,
- Постоянная актуальность требований, команда делает то что нужно заказчику,
- Постоянная корректировка под изменяющиеся требования рынка,
- Заказчик привыкает к продукту в процессе разработки,
- Низкий ТТМ, продукт появляется на рынке на стадии прототипа,
- Частые релизы, актуализация продукта,
- Код - и есть документация.

Минусы:

- Короткая подготовка,
- Низкая прогнозируемость результата,
- Непредсказуемость сроков исполнения,
- Высокая нагрузка на разработчиков.



繰り返し

Обсуждаем:

- Груминг (пред-планирование) - подготовка задач к следующему циклу,
- Планирование - оценка сложности задач и объема работы на следующий цикл,
 - Stand Up (планерка) - контроль выполнения плана,
- **Ретроспектива** - анализ процессов прошедшего цикла, модификация и адаптация для улучшения процесса разработки.

Делаем:

- User story - “как пользователь я бы хотел чтобы А делало Б”. Все US обязаны нести value пользователю,
- Spike - “как разработчик я бы хотел чтобы А делало Б”. Нулевое ROI, необходимая подготовка для, дальнейшего развития продукта,
- Task - атомарная часть US,
- Feature - набор US объединенных общим функционалом,
- Epic - набор фич объединенных одним направлением.

Оцениваем:

- Story point - основная единица оценки user story. Оценка сложности задачи или рисков внезапных событий,
- SP - относительная и НЕ аддитивная оценка. За минимум принимают самую простую задачу,
- Шкала SP - числа Фибоначчи, или не математические сущности (размер футболок XS, S, M, L, XL).

Инструмент 改善 Kaizen - Ретроспектива. Без нее команда играет в Wagile.



戦い

Scrum

- Инкрементальный подход,
- Планирование итерации продукта,
- Ограниченный бэклог на итерацию,
- Ограничение сроков релиза и спринта.



Kanban

- Максимальная скорость имплементации продукта,
- Визуализация всего продуктного бэклога,
- Взятие задач по готовности,
- Планирование по необходимости,
- Релиз по мере наполненности.



役割

- **(agile) Девелопер** - разработчик и валидатор созданного продукта
- **(agile) SCRUM master/Team coach** - контроль процессов команды, следит за настроением членов команды,
- **(agile) Product Owner/Manager** - он же бизнес аналитик, формирует видение продукта, общается с заказчиками
- **Architector** - отвечает за структуру продукта, взаимосвязи модулей, расширение продукта.
- **Тимлид** - определяет приоритезированный список задач и методологию реализации, следит за выполнением технической части планов и подстраивает планы под обстоятельства
- **Инженер безопасности** - проверка соответствия лицензиям,
- **Менеджер** - отвечает за оценку команды, формирует и согласовывает планы с бизнесом, интерфейс между бизнесом и командой.

救う

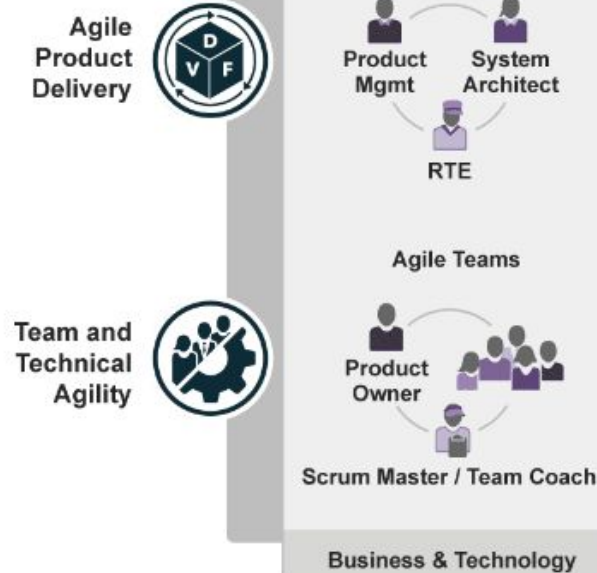
Scaled Framework (SAFe)

Agile в больших командах,

- Scrum,
- Kanban,
- Lean.

Координация на уровне менеджмента:

- Общая цель,
- Общий продукт,
- Независимые компоненты, отсутствие пересечения по планам,
- Каждая команда agile полная.



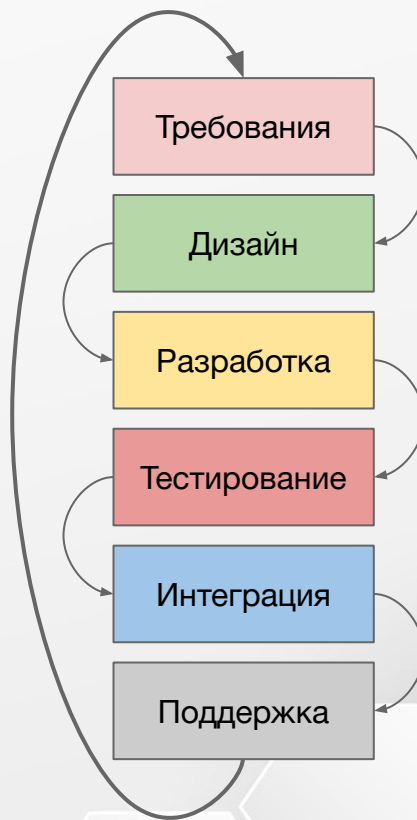


Относительно новое направление Agile методологий (2020)

- Отсутствие процесса,
- Отсутствие бюрократии,
- Отсутствие управления,
- Процесс без процесса,
- Есть только продукт...



Жизненный цикл ПО



Несмотря на разницу процессов, жизненный цикл остался каскадным.

Различные процессы определяют только величину периода.

A decorative pattern of overlapping hexagons in various shades of gray, arranged in a honeycomb-like structure on the left side of the slide.

THANK YOU!